Notas de aula - MAT01050

Leonardo Suarez

28 de abril de 2026

Notas de aula da disciplina MAT01050 - Álgebra Matricial Computacional A, oferecida no semestre letivo 26/1. A bibliografia básica é composta por *Numerical Linear Algebra with Application Using MATLAB* de W. Ford, *Numerical Linear Algebra and Applications* de B. N. Datta, *Matrix Analysis and Applied Linear Algebra* de C. D. Meyer e *Algoritmos numéricos* de Campos Filho e Frederico Ferreira. O curso será ministrado pela profa. Liliane Basso Barichello e as aulas serão às terças e quintas-feiras, das 13:30 às 15:10.

O conteúdo programático é dividido em duas áreas. Primeiro, serão introduzidos os conceitos de normas matriciais, condicionamento de problemas e estabilidade de algoritmos. Em seguida, estudaremos métodos de solução de sistemas de equações lineares: eliminação gaussiana, fatoração LU e fatoração de Cholesky. Por fim, aboraremos transformações ortogonais, em especial a decomposição QR, as rotações de Givens e as reflexões de Householder. Na segunda área, usaremos as transformações ortogonais para resolver problemas de mínimos quadrados. Seguimos com o estudo de autovalores e autovetores. A disciplina se encerra com a introdução aos métodos iterativos: métodos estacionários de primeira ordem (Jacobi, Gauss-Seidel, SOR) e métodos de projeção (gradiente, gradientes-conjugados).

Teremos uma avaliação para cada área, com atividade presencial e entrega de tarefas ao decorrer do semestre letivo. A primeira avaliação será no dia 28 de abril/26 e a segunda no dia 30 de junho/26. Uma recuperação está prevista para o dia 07 de junho/26.

Conteúdo

I Aritmética de Ponto Flutuante **3**
1 Representação de inteiros 3
2 Representação de ponto flutuante 3
3 Aritmética de ponto flutuante 3
4 Minimizando erros 3

II Normas Vetoriais e Matriciais **3**
1 Normas vetoriais 3
2 Normas matriciais 4
3 Normas matriciais submultiplicativas 5

III Condicionamento de Problemas e Estabilidade de Algoritmos **5**
1 Erros computacionais 5
2 Condicionamento de um problema 5
3 Análise de perturbação para sistemas lineares 5
4 Propriedades do número de condicionamento de uma matriz 6

IV Sistemas de Equações Lineares **6**
1 Decomposição LU 6

2 Matrizes elementares 6
3 Eliminação gaussiana com pivotamento parcial 7
4 Custo computacional da decomposição LU 8
5 Estabilidade e eficiência da eliminação gaussiana 9

V Alguns Sistemas Especiais 9
1 Sistemas tridiagonais 9
2 Matrizes simétricas positiva-definidas 10
3 Fatoração de Cholesky 11

VI Ortonormalização de Gram-Schmidt 13
1 O processo de Gram-Schmidt 13
2 Estabilidade numérica de Gram-Schmidt 15
3 A decomposição QR 16
4 Aplicações da decomposição QR 18

VII Transformações Ortogonais 19
1 Rotações de Givens 19
2 Reflexões de Householder 23

VIII A Decomposição em Valores Singulares (SVD) 27
1 O Teorema SVD 27
2 Usando a SVD para determinar propriedades de uma matriz 27
3 Compressão de imagem usando a SVD 27

IX Problemas de mínimos quadrados 27
1 Existência e unicidade de soluções de mínimos quadrados 28
2 Resolvendo problemas de mínimos quadrados sobredeterminados 29
3 Condicionamento de problemas de mínimos quadrados 29

X Autovalores e Autovetores 29
1 Teoremas de Gerschgorin 29
2 Método da potência para computar autovalor dominante 29
3 Iteração inversa de Hessenberg 29

XI Métodos Iterativos 29
1 Método de Jacobi 29
2 Método de Gauss-Seidel 29
3 Iteração SOR 29
4 Convergência dos métodos iterativos 29

Listings

1 Decomposição LU com pivotamento parcial 7
2 Resolução de sistema linear usando decomposição LU 8
3 Decomposição LU de matriz tridiagonal 9
4 Resolução de sistema linear tridiagonal usando decomposição LU 9
5 Fatoração de Cholesky 12
6 Gram-Schmidt clássico 15
7 Gram-Schmidt modificado 15

8 Decomposição QR usando Gram-Schmidt modificado 17
9 Multiplicação de uma matriz por uma matriz de Givens 22
10 Cálculo dos parâmetros de Givens 22
11 Decomposição QR de Givens 23
12 Zerando entradas na primeira coluna de uma matriz usando uma reflexão de Householder 26
13 Decomposição QR de Householder 27

I Aritmética de Ponto Flutuante

1 Representação de inteiros

2 Representação de ponto flutuante

3 Aritmética de ponto flutuante

4 Minimizando erros

II Normas Vetoriais e Matriciais

1 Normas vetoriais

Definição 1. Uma função $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ é uma *norma vetorial* dado que:

1. $\|x\| \geq 0$ para todo $x \in \mathbb{R}^n$ e $\|x\| = 0$ se e somente se $x = 0$ (*positividade*);
2. $\|\alpha x\| = |\alpha|\|x\|$ para todo $\alpha \in \mathbb{R}$ (*homogeneidade*);
3. $\|x + y\| \leq \|x\| + \|y\|$ para todo $x, y \in \mathbb{R}^n$ (*desigualdade triangular*).

Define-se a *norma-p* como

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p},$$

para $p \geq 1$. Os valores $p = 1, 2, \infty$[1] são as normas mais utilizadas, onde

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|,$$
$$\|x\|_2 = \left(\sum_{i=1}^{n} |x_i|^2 \right)^{1/2},$$
$$\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|.$$

Lema (normas são equivalentes).

$$\|x\|_\infty \leq \|x\|_2 \leq \sqrt{n}\|x\|_\infty,$$
$$\|x\|_2 \leq \|x\|_1 \leq \sqrt{n}\|x\|_2,$$
$$\|x\|_\infty \leq \|x\|_1 \leq n\|x\|_\infty.$$

[1] As normas $p = 2$ e $p = \infty$ são chamadas de *norma euclidiana* e *norma do máximo*, respectivamente.

A norma-2 é a mais usada em aplicações, pois goza diversas propriedades algébricas e geométricas como a *desigualdade de Cauchy-Schwarz*:

$$|\langle x, y\rangle| \leq \|x\|_2 \|y\|_2,$$

com a igualdade ocorrendo quando x e y são colineares. Outra relação que envolve a norma euclidiana é o *Teorema de Pitágoras* para x e y ortogonais:

$$\|x + y\|_2^2 = \|x\|_2^2 + \|y\|_2^2.$$

Por fim, a norma-2 é *ortogonalmente invariante*, isto é, para qualquer matriz P de ordem n ortogonal[2] vale

$$\|Px\|_2 = \|x\|_2,$$

para todo $x \in \mathbb{R}^n$. Sabendo que as colunas $p_1, \ldots p_n$ de P ortogonal formam base ortonormal[3] de $\mathbb{R}^n$, a propriedade acima indica que o comprimento de x não se altera na mudança de base da base canônica (que é ortonormal) para a base ortonormal $p_1, \ldots p_n$.

2 Normas matriciais

Usamos normas vetoriais para mensurar o comprimento de um vetor. Similarmente, definimos normas matriciais para medir o tamanho de matrizes. O tamanho de uma matriz é usado para avaliar se a solução x de um sistema linear $Ax = b$ pode ser confiável, para analisar a taxa de convergência de uma sequência de vetores etc. A definição de norma matricial é análoga à definição de norma vetorial, mas agora a função $\|\cdot\|$ é definida sobre o espaço de matrizes $\mathbb{R}^{m\times n}$.

Definição 2. Uma função $\|\cdot\| : \mathbb{R}^{m\times n} \to \mathbb{R}$ é uma *norma matricial* se satisfaz as seguintes propriedades:

1. $\|A\| \geq 0$ para toda $A \in \mathbb{R}^{m\times n}$ e $\|A\| = 0$ se e somente se $A = 0$ (*positividade*);
2. $\|\alpha A\| = |\alpha|\|A\|$ para todo $\alpha \in \mathbb{R}$ (*homogeneidade*);
3. $\|A + B\| \leq \|A\| + \|B\|$ para todo $A, B \in \mathbb{R}^{m\times n}$ (*desigualdade triangular*).

Uma das normas matriciais mais simples é a *norma de Frobenius*, ou também *norma de Hilbert-Schmidt*. Ela é definida como a raiz quadrada da soma dos quadrados de todas as entradas da matriz, isto é,

$$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}|^2}.$$

A classe de normas matriciais mais usadas são as *normas matriciais induzidas*, que são definidas em termos de uma norma vetorial.

Definição 3. Seja $\|\cdot\|$ uma norma vetorial, A uma matriz $m \times n$ e x um vetor $n \times 1$. Então a *norma matricial induzida* por $\|\cdot\|$ é dada por

$$\|A\| = \max_{x\neq 0} \frac{\|Ax\|}{\|x\|} = \max_{\|x\|=1} \|Ax\|. \tag{1}$$

[2]Uma matriz ortogonal (ou ortonormal) Q é uma matriz real simétrica cujas linhas e colunas são vetores ortonormais, ou seja, $Q^TQ = QQ^T = I$.

[3]Prova-se que P é ortogonal sse suas colunas são ortonormais. Daí se $p_1, \ldots, p_k \in \mathbb{R}^n$ são ortogonais então formam base dum subespaço de dimensão k de $\mathbb{R}^n$

3 Normas matriciais submultiplicativas

III Condicionamento de Problemas e Estabilidade de Algoritmos

1 Erros computacionais

2 Condicionamento de um problema

Definição 1. Um problema é *mal-condicionado* se uma pequena perturbação nos dados de entrada pode[4] causar uma grande perturbação na solução, independentemente do algoritmo usado para resolver o problema. Se pequenas perturbações nos dados de entrada causam pequenas perturbações na solução, dizemos que o problema é *bem-condicionado*.

Definição 2. Sejam x e $\overline{x}$ as soluções exata e aproximada, e $f(x)$ e $f(\overline{x})$ as respectivas soluções. Então,

- O problema é *bem-condicionado* com respeito a x se sempre que $|x - \overline{x}|$ é pequeno, $|f(x) - f(\overline{x})|$ é pequeno;
- O problema é *mal-condicionado* com respeito a x se existe uma perturbação arbitrariamente pequena $|x - \overline{x}|$ tal que $|f(x) - f(\overline{x})|$ é grande.

Definição 3. O *número de condicionamento* de um problema f com entrada x é

$$C_f(x) \stackrel{\text{def}}{=} \lim \varepsilon \to^+ \sup_{\delta x \leq \varepsilon} \frac{\frac{\|f(x+\delta x) - f(x)\|}{\|f(x)\|}}{\frac{\|\delta x\|}{\|x\|}} \tag{2}$$

Observação. O *supremo* (sup) de um subconjunto S de um conjunto ordenado X é o menor elemento de X que é maior ou igual a todos os elementos de S. O supremo difere do máximo (max) de um conjunto S porque o supremo pode não pertencer a S.

Se f é diferenciável, podemos descrever $C_f(x)$ em termos de f' usando o teorema do valor médio.

Teorema 1 (teorema do valor médio). *Se uma função $f : [a, b] \to \mathbb{R}$ é contínua em $[a, b]$ e diferenciável em (a, b), então existe um ponto $\xi \in (a, b)$ tal que*

$$f'(\xi) = \frac{f(b) - f(a)}{b - a}.$$

Fixa x e faz $\overline{x} = x + \delta x$. Aplica (1) e obtém $C_f(x) = |x||f'(x)|/|f(x)|$.

3 Análise de perturbação para sistemas lineares

Teorema 2. *Sejam A uma matriz não singular, $b \neq 0$ um vetor, x a solução para $Ax = b$ e $\|\cdot\|$ uma norma subordinada.*

1. *Se $x + \delta x$ é solução para o sistema perturbado $A(x + \delta x) = b + \delta b$, então*

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\|\frac{\|\delta b\|}{\|b\|}. \tag{3}$$

2. *Se $x + \delta x$ é solução para o sistema perturbado $(A + \delta A)(x + \delta x) = b$, então*

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq \|A\|\|A^{-1}\|\frac{\|\delta A\|}{\|A\|}. \tag{4}$$

[4] Um problema mal-condicionado ainda assim pode ter solução satisfatória.

3. *Se $x + \delta x$ é solução para o sistema perturbado $(A + \delta A)(x + \delta x) = b + \delta b$, então*

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\| \left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|A\|\|x + \delta x\|} \right) \tag{5}$$

Definição 4. O número $\|A\|\|A^{-1}\|$ é chamado de *número de condicionamento* de A e é denotado por $\kappa(A)$.

Observação. Qualquer norma matricial pode ser usada para computar o número de condicionamento. As notações $\kappa_\infty(A)$, $\kappa_1(A)$ e $\kappa_F(A)$ se referem às normas $\infty-$, $1-$ e $F-$, respectivamente.

Definição 5. Se o número de condicionamento da matriz A é grande[5], dizemos que A é *mal-condicionada*; caso contrário, A é *bem-condicionada*.

4 Propriedades do número de condicionamento de uma matriz

IV Sistemas de Equações Lineares

1 Decomposição LU

Por ora, sem considerar trocas de linhas, podemos fatorar uma matriz A de ordem n no produto de uma matriz triangular-inferior L e uma matriz triangular-superior U. Faz $L = I$. Aplica a eliminação gaussiana em A para obter a matriz triangular-superior U. Sendo a_{ii} o elemento pivô, coloca os multiplicadores $a_{i+1,i}/a_{ii}, a_{i+2}/a_{ii}, \ldots, a_{ni}/a_{ii}$, com $1 \leq i \leq n-1$, nas posições $(i+1, i), (i+2, i), \ldots, (n, i)$ da matriz L. Ao final da eliminação, tem-se $A = LU$.

Dada a decomposição $A = LU$, podemos resolver o sistema $Ax = b$. Substitui A por LU e fica

$$LUx = b \tag{6}$$
$$L(Ux) = b. \tag{7}$$

Faz $y = Ux$ e resolve $Ly = b$ usando substituição para frente (*forward substitution*). Depois resolve $Ux = y$ com substituição para trás (*back substitution*).

2 Matrizes elementares

Definição 1. Para cada uma das três operações elementares de linha, existe uma *matriz elementar* E_{ij}, $E_i(t)$ e $E_{ij}(t)$:

1. E_{ij} é a matriz que troca as linhas i e j da matriz identidade I;
2. $E_i(t)$ é a matriz que multiplica a linha i da matriz identidade I por um escalar $t \neq 0$;
3. $E_{ij}(t)$ é a matriz que subtrai t vezes a linha j da linha i da matriz identidade I.

Proposição 1. *Se uma matriz $A_{n\times n}$ é multiplicada à esquerda por uma matriz elementar E, então o resultado é a matriz obtida pela aplicação da operação elementar correspondente à E nas linhas da matriz A.*

Proposição 2. *Matrizes elementares são não singulares; de fato*

1. $E_{ij}^{-1} = E_{ij}$;

[5] Imediatamente se deve perguntar o que é um número de condicionamento grande. A resposta depende do problema analisado.

2. $E_i(t)^{-1} = E_i(1/t)$;

3. $E_{ij}(t)^{-1} = E_{ij}(-t)$.

Proposição 3. *Se B é linha-equivalente a A, então A é linha-equivalente a B.*

Proposição 4. *Seja $A_{n\times n}$ uma matriz não singular. Então*

1. *A é linha-equivalente à matriz identidade I;*

2. *A é produto de matrizes elementares.*

Proposição 5. *Seja $A_{n\times n}$ uma matriz e suponha que A é linha-equivalente a I. Então A é não singular e A^{-1} pode ser obtida por meio da aplicação das mesmas operações elementares usadas para transformar A em I.*

Proposição 6. *A decomposição LU de uma matriz $A_{n\times n}$ não singular é única.*

3 Eliminação gaussiana com pivotamento parcial

O algoritmo da eliminação gaussiana pode gerar problemas se os elementos pivôs são pequenos. Os erros ocorrem quando dividimos por elementos pivôs pequenos e obtemos multiplicadores a_{ji}/a_{ii} grandes. Esses multiplicadores, quando somados a elementos pequenos, resultam numa perda de contribuição dos números menores. A solução para esse problema é escolher o elemento com maior magnitude em cada coluna como pivô, de modo que $|a_{ji}/a_{ii}| \leq 1$ para $i+1 \leq j \leq n$. Chamamos essa estratégia de *eliminação gaussiana com pivotamento parcial* (EGPP).

Se usarmos EGPP, a decomposição LU de A consiste em três matrizes P, L e U tal que

$$PA = LU, \tag{8}$$

onde P é uma *matriz de permutação*, ou *matriz pivô*, que representa as trocas de linha feitas durante o processo de eliminação.

O seguinte código realiza a decomposição LU com pivotamento parcial:

Listing 1: Decomposição LU com pivotamento parcial

```
function [L,U,P] = ludecomp(A)
    // performs LU decomposition using gaussian elimination with partial
        pivoting
    // input: n x n matrix A
    // output: lower-triangular matrix L, upper-triangular matrix U, and
        permutation matrix P such that PA=LU
    n = size(A,1);
    U = A;
    L = eye(n,n);
    P = eye(n,n);
    for i = 1:n-1
        [m,k] = max(abs(U(i:n,i))); // get the largest pivot in each
            column (value m, index k)
        pivot_index = k+i-1;
        //perform row exchanges if current row is not a pivot row
        if pivot_index ~= i then
            U([i,pivot_index],:) = U([pivot_index,i],:); // exchange rows
                i and pivot_index in matrix A
            P([i,pivot_index],:) = P([pivot_index,i],:);
```

```
            if i > 1 then
                L([i,pivot_index],1:i-1) = L([pivot_index,i],1:i-1); //
                    exchange rows before the current column in matrix L
            end
        end
        multipliers = U(i+1:n,i) / U(i,i);
        U(i+1:n,i+1:n) = U(i+1:n,i+1:n) - multipliers*U(i,i+1:n); //
            update submatrix
        L([i+1:n],i) = multipliers;
    end
    U = triu(U);
endfunction
```

4 Custo computacional da decomposição LU

Se o sistema linear $Ax_i = b_i$, $1 \leq i \leq k$, precisa ser resolvido para múltiplos vetores b_i, podemos realizar a decomposição $PA = LU$, de custo computacional $\mathcal{O}(n^3)$, apenas uma vez. Depois resolvemos $L(Ux_i) = Pb_i$ usando substituição para frente e substituição para trás, que têm custo $\mathcal{O}(kn^2)$. Caso fôssemos realizar a eliminação gaussiana para cada sistema $Ax_i = b_i$, o custo seria $\mathcal{O}(kn^3)$.

Dada uma decomposição LU de uma matriz A, podemos resolver o sistema $Ax = b$ da seguinte forma:

Listing 2: Resolução de sistema linear usando decomposição LU

```
function [x] = lusolve(L,U,P,b)
    // solve linear system given LU decomposition
    // input: lower-triangular matrix L, upper-triangular matrix U,
        permutation matrix P, and vector b
    // output: solution vector x
    n = size(b,1);

    b = P*b; // perform same row exchanges from LU factorization in the
        vector b
    // solve Ly = b
    y = zeros(n,1); //initialize vector y
    y(1) = b(1);
    for i = 2:n
        y(i) = (b(i) - L(i,1:i-1)*y(1:i-1)) / L(i,i); // perform forward
            substitution
    end
    // solve Ux = y
    x = zeros(n,1); // initialize solution vector x
    x(n) = y(n) / U(n,n);
    for k = n-1:-1:1
        x(k) = (y(k) - U(k,k+1:n)*x(k+1:n)) / U(k,k); // perform back
            substitution
    end
endfunction
```

5 Estabilidade e eficiência da eliminação gaussiana

Definição 2. Aplica a fatoração LU na matriz A. Durante os passos da eliminação, temos as matrzes $A = A^{(0)}, A^{(1)}, \ldots, A^{(n-1)} = U$. O *fator de crescimento* é a razão

$$\rho \stackrel{\text{def}}{=} \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|},$$

em outras palavras, é a razão entre o maior elemento que aparece durante a eliminação gaussiana e o maior elemento da matriz A.

V Alguns Sistemas Especiais

1 Sistemas tridiagonais

Uma matriz T tridiagonal[6] pode ser fatorada como produto de duas matrizes bidiagonais L e U. A matriz L é unitária triangular-inferior e a matriz U é triangular-superior, com elementos da superdiagonal iguais aos elementos da superdiagonal de T. O programa abaixo realiza a decomposição LU de uma matriz tridiagonal T:

Listing 3: Decomposição LU de matriz tridiagonal

```
function [l,u] = tridiaglu(a,b,c)
    // factor tridiagonal matrix T defined by subdiagonal a, main
        diagonal b, and superdiagonal c
    // into a product of two bidiagonal matrices L (with 1's on the main
        diagonal) and U (whose superdiagonal elements are
    // the same as c)
    // input: vectors a, b, c
    // output: subdiagonal l of the left bidiagonal factor and main
        diagonal u of the right bidiagonal factor
    u(1) = b(1);

    for i = 1:n-1
        l(i) = a(i) / u(i);
        u(i+1) = b(i+1) - l(i)*c(i);
    end
endfunction
```

Observação. Deve-se lembrar que o código acima não funciona se $u(i) = 0$ para algum i, mas isso raramente acontece na prática. Infelizmente, a estabilidade do algoritmo não pode ser garantida.

Para resolver um sistema $Tx = b$ com T tridiagonal, basta realizar a decomposição LU e depois resolver $Ly = b$ e $Ux = y$ usando substituição para frente e substituição para trás, respectivamente, conforme o código abaixo:

Listing 4: Resolução de sistema linear tridiagonal usando decomposição LU

```
function [x] = trisolve(l,u,c,rhs)
    // solve Tx=b given tridiagonal factorization of T
    // into unit lower-bidiagonal matrix L and upper-bidiagonal matrix U
    // input: l is the subdiagonal of the lower-diagonal matrix, u is the
        main diagonal of upper-diagonal matrix
```

[6] T é uma matriz *tridiagonal* se $t_{ij} = 0$ sempre que $|i - j| > 1$.

```
        // output: solution vector x
        [n,m] = size(rhs);

        // solve Ly=rhs by forward substitution
        y = zeros(n,1);
        y(1) = rhs(1);
        for i = 2:n
            y(i) = rhs(i) - l(i-1)*y(i-1);
        end

        // solve Ux=y by back substitution
        x = zeros(n,1);
        x(n) = y(n) / u(n);
        for j = n-1:-1:1
            x(j) = (y(j) - c(j)*x(j+1)) / u(j);
        end
    endfunction
```

O algoritmo acima tem custo computacional $\mathcal{O}(n)$. No programa listing 3, o laço `for` é executado $n-1$ vezes, com cada iteração realizando uma divisão, uma subtração e uma multiplicação, então a contagem de flops é $3(n-1)$. A substituição para frente requer $2(n-1)$ flops (uma subtração e uma multiplicação) e a substituição para trás requer $1+3(n-1)$ flops (a divisão inicial $x_n = y_n/u_n$, seguida de $n-1$ iterações, cada uma com uma subtração, uma multiplicação e uma divisão). Assim, o número total de flops para resolver um sistema linear tridiagonal $Tx = b$ usando a decomposição LU é $3(n-1)+2(n-1)+1+3(n-1) = 8n-7$.

2 Matrizes simétricas positiva-definidas

Definição 1. Uma matriz simétrica A é *positiva definida* se $x^TAx > 0$ para todo $x \in \mathbb{R}^n$, $x \neq 0$. A expressão $x^TAx = \sum_{i=1}^n \sum_{j=1}^n a_{ij}x_ix_j$ é chamada de *forma quadrática* associada à matriz A. Se $x^TAx \geq 0$ para todo $x \in \mathbb{R}^n$, $x \neq 0$, dizemos que A é *positiva semi-definida.*

Observação. Nestas notas, todas as matrizes positivas definidas serão também simétricas, então usaremos apenas o termo positiva definida. É possível que uma matriz não simétrica satisfaça

$$x^TAx > 0, \quad x \neq 0$$

mas não existe um consenso sobre o que isso significa para matrizes não simétricas.

A condição $x^TAx > 0$ para todo $x \neq 0$ é quase impossível de verificar na prática, mas existem outras condições equivalentes que determinam se uma matriz é positiva definida, como mostra o teorema abaixo.

Teorema 1.

1. *Uma matriz simétrica $A_{n\times n}$ é positiva definida se e somente se todos os seus autovalores são positivos.*
2. *Uma matriz simétrica $A_{n\times n}$ é positiva definida se e somente se todos os seus menores principais líderes são positivos, isto é,* $\det A(1:i, 1:i) > 0$ *para* $1 \leq i \leq n$. *(critério de Sylvester).*
3. *Se $A = (a_{ij})$ é positiva definida então $a_{ii} > 0$ para todo i.*
4. *Se $A = (a_{ij})$ é positiva definida então o maior elemento em magnitude de todas as entradas da matriz está na diagonal, ou seja,* $\max_{i,j} |a_{ij}| = \max_i |a_{ii}|$.

5. A soma de duas matrizes positivas definidas é positiva definida.

Nota que, enquanto os itens 1 e 2 caracterizam matrizes positivas definidas, os itens 3 e 4 nos permitem dizer rapidamente se uma matriz *não* é positiva definida, i.e., são condições necessárias, mas não suficientes.

Demonstração. 1. ($\Rightarrow$).

2. ($\Rightarrow$).

3. c

4. d

5. e

□

Observação. Se A é uma matriz invertível então A^TA é positiva definida. De fato,

$$x^TA^TAx = (Ax)^T(Ax) = \|Ax\|_2^2 > 0, \quad x \neq 0.$$

Observação. Uma matriz é *negativa definida* se $-A$ é positiva definida. Uma matriz é *simétrica indefinida* se tem autovalores positivos e negativos, ou seja, se existe x tal que $x^TAx > 0$ e existe y tal que $y^TAy < 0$.

Teorema 2. *Suponha que uma matriz real simétrica tridiagonal* $A = \begin{bmatrix} b_1 & a_1 & 0 & \dots & 0 \\ a_1 & b_2 & a_2 & \dots & 0 \\ 0 & a_2 & b_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & a_{n-1} \\ 0 & 0 & 0 & a_{n-1} & b_n \end{bmatrix}$ *com entradas na diagonal positivas é estritamente diagonal dominante, i.e.,* $b_i > |a_{i-1}| + |a_i|$ *para* $1 \leq i \leq n$. *Então* A *é positiva definida.*

3 Fatoração de Cholesky

Mostrar que os autovalores ou que os menores principais líderes de uma matriz A são positivos pode ser difícil. A fatoração de Cholesky é um método eficiente para determinar se uma matriz é positiva definida e, caso seja, obter uma fatoração $A = R^TR$, onde R é triangular-superior. A existência e unicidade dessa fatoração é garantida pelo seguinte teorema:

Teorema 3. *Seja* $A_{n\times n}$ *uma matriz real positiva definida. Então existe uma única matriz triangular-superior* $R = (r_{ij})$ *com* $r_{ii} > 0, 1 \leq i \leq n$, *tal que*

$$A = R^TR.$$

O esquema a seguir usa o caso $n = 3$ para ilustrar o algoritmo da fatoração de Cholesky:

Computando a fatoração de Cholesky

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix} = \begin{bmatrix} r_{11} & 0 & 0 \\ r_{12} & r_{22} & 0 \\ r_{13} & r_{23} & r_{33} \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix} = \begin{bmatrix} r_{11}^2 & r_{11}r_{12} & r_{11}r_{13} \\ r_{11}r_{12} & r_{12}^2 + r_{22}^2 & r_{12}r_{13} + r_{22}r_{23} \\ r_{11}r_{13} & r_{12}r_{13} + r_{22}r_{23} & r_{13}^2 + r_{23}^2 + r_{33}^2 \end{bmatrix} \quad (9)$$

Igualando as entradas das matrizes, tem-se

$$\begin{aligned} r_{11} &= \sqrt{a_{11}}, \\ r_{12} &= a_{12}/r_{11}, \\ r_{13} &= a_{13}/r_{11}, \\ r_{22} &= \sqrt{a_{22} - r_{12}^2}, \\ r_{23} &= (a_{23} - r_{12}r_{13})/r_{22}, \\ r_{33} &= \sqrt{a_{33} - r_{13}^2 - r_{23}^2}. \end{aligned}$$

A decomposição de Cholesky pode ser usada não só para fatorar A, como também para determinar se A é positiva definida. Nota que A é positiva definida somente se $a_{ii} > 0$ para $1 \leq i \leq n$. No entanto, se o argumento de alguma raiz quadrada (como $a_{22} - r_{12}^2$) for negativo, então A não é positiva definida. Implementamos a seguir a fatoração de Cholesky:

Listing 5: Fatoração de Cholesky

```
function [R] = cholesky(A)
    // factor positive definite matrix A using Cholesky decomposition
    // if the algorithm fails, A is not positive definite
    // input: n x n symmetric matrix A
    // output: error message with empty matrix or upper-triangular matrix
        R
    [n, m] = size(A);
    if n ~= m then
        error('Input␣matrix␣must␣be␣square');
    end

    R = zeros(n,n); // initialize R

    for i = 1:n // traverse matrix rows
        sum_squares = 0;
        for k = 1:i-1
            sum_squares = sum_squares + R(k,i)^2;
        end
        tmp = A(i,i) - sum_squares;
        if tmp <= 0 then
            disp('Input␣matrix␣is␣not␣positive␣definite');
            R = []; // return empty matrix
            return
```

```
            else
                R(i,i) = sqrt(tmp);
            end
            for j = i+1:n // traverse (n-i) matrix columns
                sum_product = 0;
                for k = 1:i-1
                    sum_product = sum_product + R(k,i)*R(k,j);
                end
                R(i,j) = (A(i,j) - sum_product) / R(i,i);
            end
        end
    endfunction
```

Pode ser mostrado que o algoritmo da fatoração de Cholesky tem contagem de flops de $n^3/3 + \mathcal{O}(n^2)$, que é aproximadamente metade do custo da decomposição LU, que tem contagem de flops de $2n^3/3 + \mathcal{O}(n^2)$. Assim, a fatoração de Cholesky é mais eficiente do que a decomposição LU para matrizes positivas definidas. Também pode ser mostrado que o algoritmo de Cholesky é estável para trás.

Se A é positiva definida, a decomposição de Cholesky pode ser usada para resolver o sistema linear $Ax = b$. Com efeito, se $A = R^T R$, então $R^T Rx = b$. Faz $y = Rx$ e resolve $R^T y = b$ usando substituição para frente. Depois resolve $Rx = y$ usando substituição para trás. Como cada uma das substituições para frente e para trás tem custo $\mathcal{O}(n^2)$, o custo total para resolver $Ax = b$ usando a fatoração de Cholesky é aproximadamente $n^3/3 + n^2$.

VI Ortonormalização de Gram-Schmidt

1 O processo de Gram-Schmidt

Dado um conjunto de vetores linearmente independentes $S = \{v_1, v_2, \ldots, v_n\} \in \mathbb{R}^m$, o *processo de Gram-Schmidt* (GS) constrói um conjunto de vetores ortonormais $S' = \{e_1, e_2, \ldots, e_n\}$ que geram o mesmo subespaço n-dimensional de S. Segue que $m \geq n$, caso contrário os vetores seriam linearmente dependentes.

Antes de descrever o processo de GS, definimos o operador de projeção ortogonal, um dos usos mais importantes do produto interno.

Definição 1. A projeção ortogonal de um vetor v sobre um vetor u é dada pela operador

$$\text{proj}(v)_u = \frac{\langle v, u \rangle}{\langle u, u \rangle} u.$$

Aqui, a ideia crucial é que se removermos a projeção ortogonal de v sobre u de v por meio da operação $v - \text{proj}(v)_u$, obtemos um vetor que é ortogonal a u. Podemos verificar esse fato algebricamente:

$$\langle u, v - \text{proj}(v)_u \rangle = \langle u, v \rangle - \langle u, \text{proj}(v)_u \rangle = \langle u, v \rangle - \frac{\langle v, u \rangle}{\langle u, u \rangle} \langle u, u \rangle = 0.$$

O processo de GS, descrito a seguir, tem como base sucessivas remoções da projeção ortogonal de cada vetor v_i sobre os vetores $e_1, e_2, \ldots, e_{i-1}$ já construídos de $v_1, v_2, \ldots, v_{i-1}$, para obter um vetor u_i ortogonal a todos os vetores e_j com $j < i$. Depois, o vetor u_i é normalizado para obter o vetor e_i.

Processo de Gram-Schmidt

1. Toma o primeiro vetor v_1 e normaliza para obter $e_1 = v_1/\|v_1\|$ e define $r_{11} = \|v_1\|_2$. Usaremos os elementos do conjunto $\{r_{ij}, 1 \leq i, j \leq n, j \geq i\}$ para construir uma matriz triangular superior R que, junto com a matriz Q de colunas e_i, formará a decomposição $A = QR$ da matriz A cujas colunas são os vetores v_i. Seja $u_1 = v_1$ e então

$$e_1 = \frac{v_1}{\|v_1\|_2} = \frac{v_1}{r_{11}}, \quad r_{11} = \|v_1\|_2.$$

2. Seja $r_{12} = \langle v_2, e_1 \rangle$. Defina o vetor u_2, normal a e_1, removendo a projeção de v_2 sobre e_1. Então normaliza u_2 para obter e_2:

$$u_2 = v_2 - \text{proj}(v_2)_{e_1} = v_2 - \frac{\langle v_2, e_1 \rangle}{\|e_1\|_2^2} e_1 = v_2 - r_{12} e_1.$$

   Normaliza o vetor u_2. Seja $r_{22} = \|u_2\|_2$ e define

$$e_2 = \frac{u_2}{\|u_2\|_2} = \frac{u_2}{r_{22}}.$$

3. Para obter u_3, normal a e_1 e e_2, remove as projeções de v_3 sobre e_1 e e_2, definindo $r_{13} = \langle v_3, e_1 \rangle$ e $r_{23} = \langle v_3, e_2 \rangle$:

$$u_3 = v_3 - \text{proj}(v_3)_{e_1} - \text{proj}(v_3)_{e_2} = v_3 - r_{13} e_1 - r_{23} e_2.$$

   Normaliza o vetor u_3, definindo $r_{33} = \|u_3\|_2$ para obter

$$e_3 = \frac{u_3}{\|u_3\|_2} = \frac{u_3}{r_{33}}.$$

4. Para $4 \leq i \leq n$, define $r_{ji} = \langle v_i, e_j \rangle$ para $1 \leq j < i$ e então

$$u_i = v_i - \sum_{j=1}^{i-1} \text{proj}(v_i)_{e_j} = v_i - \sum_{j=1}^{i-1} r_{ji} e_j.$$

   Seja $r_{ii} = \|u_i\|_2$. Normaliza u_i para obter

$$e_i = \frac{u_i}{\|u_i\|_2} = \frac{u_i}{r_{ii}}.$$

   A sequência de vetores $e_1, e_2, \ldots, e_n$ é ortonormal e gera o mesmo subespaço que os vetores $v_1, v_2, \ldots, v_n$. Para ver isso, tome um vetor x que é uma combinação linear dos vetores v_i, isto é,

$$x = \sum_{i=1}^{n} c_i v_i.$$

   Por construção, tem-se que $v_i = u_i + \sum_{j=1}^{i-1} \text{proj}(v_i)_{e_j} e_j$ para $1 \leq i \leq n$. Mas cada $\text{proj}(v_i)_{e_j}$ é múltiplo escalar de e_j, e $u_i = \|u_i\| e_i$, então x pode ser escrito como combinação linear de $e_1, e_2, \ldots, e_n$. Como esses vetores são ortonormais, eles também são linearmente independentes, e portanto $e_1, e_2, \ldots, e_n$ formam uma base ortonormal para o mesmo subespaço gerado por $v_1, v_2, \ldots, v_n$.

Implementamos o processo de GS no algoritmo abaixo, chamado de *Gram-Schmidt clássico*.

Listing 6: Gram-Schmidt clássico

```
function [E] = clgrsch(V)
    // converts a set of linearly independent vectors to a set of
        orthonormal vectors spanning the same subspace
    // input: m x n matrix V whose columns are the LI vectors to be
        orthonormalized
    // output: m x n matrix E whose columns are an orthonormal set of
        vectors that span the same subspace as the columns of V
    [m,n] = size(V);
    for i = 1:n
        sumproj = 0;
        for j = 1:i-1
            sumproj = sumproj + (V(:,i)' * E(:,j)) * E(:,j); // compute
                sum of projections of v_i onto e_1, e_2, ..., e_{i-1}
        end
        E(:,i) = V(:,i) - sumproj; // compute u_i, orthogonal to e_1, e_2
            , ..., e_{i-1}, by removing from v_i its
        // orthogonal projections onto e_1, e_2, ..., e_{i-1}
        E(:,i) = E(:,i) / norm(E(:,i),2); // normalize u_i to obtain unit
             vector e_i
    end
endfunction
```

2 Estabilidade numérica de Gram-Schmidt

Durante a execução do processo de GS, os vetores u_i não são exatamente ortogonais devido a erros de arredondamento. Para listing 6, essa perda de ortogonalidade pode ser particularmente severa. Além disso, o algoritmo pode gerar resultados ruins quando alguns dos vetores são quase linearmente dependentes. Por conta disso, é dito que o GS clássico é numericamente instável.

Podemos melhorar a estabilidade do processo de GS usando o algoritmo de *Gram-Schmidt modificado* (MGS), em que as projeções são removidas de cada vetor v_i uma a uma, em vez de remover todas as projeções de v_i sobre os vetores e_j com $j < i$ de uma só vez. O código abaixo implementa o processo MGS:

Listing 7: Gram-Schmidt modificado

```
function [E] = modgrsch(V)
    // modified GS to convert a set of linearly independent vectors to a
        set of orthonormal vectors spanning the same subspace
    // instead of removing all projections at once, we remove one
        projection at a time to reduce rounding errors
    // input: m x n matrix V whose columns are the LI vectors to be
        orthonormalized
    // output: m x n matrix E whose columns are an orthonormal set of
        vectors that span the same subspace as the columns of V
    [m,n] = size(V);
    for i = 1:n
        E(:,i) = V(:,i);
        for j = 1:i-1
            E(:,i) = E(:,i) - (E(:,i)' * E(:,j)) * E(:,j); // remove from
                v_i its projection onto e_j one at a time
```

```
            // to obtain a vector u_i that is orthogonal to each e_j
        end
        E(:,i) = E(:,i) / norm(E(:,i),2); // normalize u_i to obtain unit
            vector e_i
    end
endfunction
```

Definição 2. Uma matriz real $A_{m\times n}$ tem *posto completo* (*full rank*) se rank $A = \min\{m, n\}$. Se $m \geq n$ e A tem posto completo, então as colunas de A são linearmente independentes. Se $m \leq n$ e A tem posto completo, então as linhas de A são linearmente independentes.

3 A decomposição QR

Suponha que $A_{m\times n} = [v_1\ v_2\ \ldots\ v_n]$ é uma matriz de posto completo. O processo de GS pode ser usado para fatorar A num produto $A = QR$, onde $Q_{m\times n}$ tem colunas ortonormais e $R_{n\times n}$ é triangular-superior. De fato, se $e_1, e_2, \ldots, e_n$ são os vetores ortonormais obtidos a partir do processo de GS aplicado aos vetores $v_1, v_2, \ldots, v_n$, então $Q = [e_1\ e_2\ \ldots\ e_n]$ é uma matriz ortogonal. Além disso, os elementos r_{ij} usados para construir os vetores e_i formam uma matriz triangular-superior R. Para ver que $A = QR$, note que $v_i = u_i + \sum_{j=1}^{i-1} r_{ji}e_j$ para $1 \leq i \leq n$, e $u_i = r_{ii}e_i$, então

$$v_i = r_{ii}e_i + \sum_{j=1}^{i-1} r_{ji}e_j, \quad 1 \leq i \leq n.$$

Sejam $Q = [e_1\ e_2\ \ldots\ e_n]$ a matriz cujas colunas são os vetores ortonormais

$$e_1 = \begin{bmatrix} e_{11} \\ e_{21} \\ \vdots \\ e_{m1} \end{bmatrix}, e_2 = \begin{bmatrix} e_{12} \\ e_{22} \\ \vdots \\ e_{m2} \end{bmatrix}, \ldots, e_n = \begin{bmatrix} e_{1n} \\ e_{2n} \\ \vdots \\ e_{mn} \end{bmatrix},$$

e

$$R = \begin{bmatrix} r_{11} & r_{12} & \ldots & r_{1n} \\ 0 & r_{22} & \ldots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & r_{nn} \end{bmatrix}.$$

É suficiente mostrar o caso $n = 3$ para ver que $A = QR$. Seja A a matriz 3×3

$$A = \begin{bmatrix} v_1 & v_2 & v_3 \end{bmatrix} = \begin{bmatrix} v_{11} & v_{12} & v_{13} \\ v_{21} & v_{22} & v_{23} \\ v_{31} & v_{32} & v_{33} \end{bmatrix}.$$

Então computa QR

$$\begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix} = \begin{bmatrix} r_{11}e_{11} & r_{12}e_{11} + r_{22}e_{12} & r_{13}e_{11} + r_{23}e_{12} + r_{33}e_{13} \\ r_{11}e_{21} & r_{12}e_{21} + r_{22}e_{22} & r_{13}e_{21} + r_{23}e_{22} + r_{33}e_{23} \\ \vdots & \vdots & \vdots \\ r_{11}e_{31} & r_{12}e_{31} + r_{22}e_{32} & r_{13}e_{31} + r_{23}e_{32} + r_{33}e_{33} \end{bmatrix}.$$

Comparando as colunas de A e QR, tem-se

$$\begin{aligned} v_1 &= r_{11}e_1, \\ v_2 &= r_{12}e_1 + r_{22}e_2, \\ v_3 &= r_{13}e_1 + r_{23}e_2 + r_{33}e_3. \end{aligned}$$

Portanto, $A = QR$.

Teorema 1. *Se $A_{m\times n}$ tem posto completo, então existe uma matriz ortogonal $Q_{m\times n}$ e uma matriz triangular-superior $R_{n\times n}$ tal que $A = QR$.*

Observação. A decomposição QR é única.

A seguir temos o código para a decomposição QR usando o processo de MGS:

Listing 8: Decomposição QR usando Gram-Schmidt modificado

```
function [Q,R] = modqrgrsch(A)
    // performs QR decomposition of a matrix using the modified GS
        procedure
    // input: m x n matrix A
    // output: m x n orthogonal matrix Q, n x n upper-triangular matrix R
    [m,n] = size(A);
    for i = 1:n
        Q(:,i) = A(:,i);
        for j = 1:i-1
            R(j,i) = Q(:,i)' * Q(:,j); // compute coefficient r_{ji} of
                the projection of v_i onto e_j
            Q(:,i) = Q(:,i) - R(j,i) * Q(:,j); // remove from v_i its
                projection onto e_j one at a time
            // to obtain a vector u_i that is orthogonal to each e_j
        end
        R(i,i) = norm(Q(:,i),2); // compute coefficient r_{ii}
        Q(:,i) = Q(:,i) / R(i,i); // normalize u_i to obtain unit vector
            e_i
    end
endfunction
```

Observação. O algoritmo listing 8 produz uma matriz $\hat{Q}$ de tamanho $m \times n$ e uma matriz $\hat{R}$ de tamanho $n \times n$. Essa fatoração é conhecida como *decomposição QR reduzida*. Se `qr` fatora A em Q de tamanho $m \times m$ e R de tamanho $m \times n$, então a fatoração é chamada de *decomposição QR completa*. Enquanto a decomposição QR reduzida produz uma base ortonormal para range A, a decomposição QR completa produz uma base ortonormal para $\mathbb{R}^m$.

Tendo listing 8 como referência, podemos determinar a contagem de flops para a decomposição QR usando o processo de MGS. O laço externo é executado n vezes. Para cada $1 \leq i \leq n$, o laço interno é executado $1, 2, \ldots, n-1$ vezes, respectivamente. Então as sentenças dentro do laço interno são executadas $1 + 2 + \ldots + n - 1 = n(n-1)/2$ vezes. Cada execução do laço interno computa um produto interno, de custo $2m$, e uma multiplicação de um vetor por um escalar e uma subtração, de custo $2m$. Depois, o laço externo executa uma norma de um vetor, de custo $2m$, e uma multiplicação de um vetor por um escalar, de custo $2m$. Assim, a contagem total de flops para a decomposição QR usando o processo de MGS é

$$\frac{n(n-1)}{2}(2m + 2m) + n(2m + 2m) = 2mn^2 + \mathcal{O}(mn).$$

Antes de discutir a estabilidade numérica da decomposição QR usando o processo de GS modificado, precisamos de uma definição para o número de condicionamento de uma matriz $m \times n$. Lembra que para uma matriz quadrada não singular, vale

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_1}{\sigma_n}, \tag{10}$$

onde σ_1 e σ_n são, respectivamente, o maior e o menor valor singular de A. Uma matriz não quadrada não possui inversa, mas possui um maior e um menor valor singular não nulos, o que leva à seguinte definição.

Definição 3. Se A é uma matriz $m \times n$, o número de condicionamento na norma-2 de A é

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_r}, \tag{11}$$

onde σ_1 e σ_r são, respectivamente, o maior e o menor valor singular não nulos de A.

Embora conceitualmente simples, a decomposição QR usando o processo de GS modificado não é estável como outros métodos que serão estudados a seguir. Isso porque a estabilidade do algoritmo está relacionada com o número de condicionamento da matriz, como mostra o teorema abaixo.

Teorema 2. *Suponha que o processo de MGS é aplicado na matriz $A_{m\times n} = [a_1\ a_2\ \ldots\ a_n]$ de posto n para obter as matrizes $\hat{Q}_{m\times n}$ e $\hat{R}_{n\times n}$. Então existem constantes c_i, $i = 1, 2, 3$, que dependem de m e n tais que*

$$A + \Delta A_1 = \hat{Q}\hat{R}, \quad \|\Delta A_1\|_2 \leq c_1 \epsilon \|A\|_2,$$
$$\|\hat{Q}^T\hat{Q} - I\|_2 \leq c_2 \epsilon \kappa_2(A) + \mathcal{O}((\epsilon\kappa_2(A))^2),$$

e existe uma matriz Q com colunas ortonormais tal que

$$A + \Delta A_2 = Q\hat{R}, \quad \|\Delta A_2(:, j)\|_2 \leq c_3 \epsilon \|a_j\|_2, \quad 1 \leq j \leq n.$$

Fazemos três observações sobre o teorema acima. Primeiro, o resíduo $A - \hat{Q}\hat{R}$ é pequeno. Segundo, o quão perto a matriz $\hat{Q}$ está de ter colunas ortonormais depende do número de condicionamento de A; se A é bem condicionada, então $\hat{Q}$ tem colunas quase ortonormais. E, por fim, $\hat{R}$ é o fator triangular-superior exato de uma matriz próxima de A.

4 Aplicações da decomposição QR

Existem inúmeras aplicações para a decomposição QR. Nesta subseção, abordamos duas aplicações simples, o cálculo do determinante e da imagem de uma matriz.

Supondo que A seja uma matriz quadrada cujas colunas são LI, podemos determinar o valor absoluto do determinante de A usando a decomposição QR. Com efeito, como Q tem colunas ortonormais,

$$Q^TQ = I \implies \det(Q^TQ) = \det(I) = 1 \implies \det(Q^T)\det(Q) = 1 \implies \det(Q)^2 = 1,$$

então $|\det(Q)| = 1$. Segue que

$$|\det(A)| = |\det(QR)| = |\det(Q)\det(R)| = |\det(R)| = \prod_{i=1}^{n} r_{ii},$$

pois o determinante de uma matriz triangular é o produto dos elementos da diagonal.

O seguinte teorema mostra como a decomposição QR pode ser usada para encontrar uma base ortonormal para a imagem de uma matriz A.

Teorema 3. *Se $A_{m\times n}$ é uma matriz de posto n e $A = QR$ é a decomposição QR com Q ortogonal e R triangular-superior, então as colunas de Q formam uma base ortonormal para a imagem de A.*

Demonstração. □

VII Transformações Ortogonais

1 Rotações de Givens

Na decomposição LU, multiplicamos uma matriz A à esquerda por uma sequência de matrizes elementares para obter $A = LU$, onde L é um produto de matrizes elementares e U é triangular-superior. Similarmente, a ideia aqui é realizar a fatoração $A = QR$ através da multiplicação à esquerda de A por uma sequência de *matrizes de Givens* e obter uma matriz ortogonal Q e uma matriz triangular-superior R. Cada multiplicação por uma matriz de Givens zera uma entrada da matriz original para construir R. O seguinte esquema ilustra o algoritmo a ser estudado:

Algoritmo da decomposição QR de Givens

Aplica $n-1$ matrizes de Givens, J_{i1}, $2 \leq i \leq n$, à esquerda de A para zerar as entradas $a_{21}, a_{31}, \ldots, a_{n1}$

$$(J_{n1}J_{n-1,1}\ldots J_{21})A \to \begin{bmatrix} X & X & \ldots & X \\ 0 & X & \ldots & X \\ 0 & X & \ldots & X \\ \vdots & \vdots & \vdots & \vdots \\ 0 & X & \ldots & X \end{bmatrix}$$

Agora usa $n-2$ matrizes de Givens para zerar os elementos nos índices $(3,2),(4,2),\ldots,(n,2)$ para obter

$$(J_{n2}J_{n-1,2}\ldots J_{42}J_{32})(J_{n1}J_{n-1,1}\ldots J_{21})A \to \begin{bmatrix} X & X & \ldots & X \\ 0 & X & \ldots & X \\ 0 & 0 & \ldots & X \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \ldots & X \end{bmatrix}$$

Seja J_i o produto de matrizes Givens usado para zerar os elementos abaixo da entrada a_{ii}. Continua o processo até obter R triangular superior

$$(J_{n-1}J_{n-2}\ldots J_2J_1)A \to \begin{bmatrix} X & X & \ldots & X & X \\ 0 & X & \ldots & X & X \\ 0 & 0 & X & \ldots & X \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & X \end{bmatrix} = R,$$

e

$$A = (J_{n-1}J_{n-2}\ldots J_2J_1)^T R = QR.$$

onde Q é o produto de matrizes ortogonais, logo ortogonal[a].

[a] Se $P = P_kP_{k-1}\ldots P_2P_1$, onde P_i é ortogonal para $1 \leq i \leq k$, então P é ortogonal, pois $P^TP = P_1^TP_2^T\ldots P^{k-1}P_k^TP_kP_{k-1}\ldots P_2P_1 = I$.

Definição 1. Uma matriz da forma

$$J(i,j,c,s) = \begin{bmatrix} 1 & 0 & 0 & \ldots & \ldots & \ldots & \ldots & 0 \\ 0 & 1 & 0 & \ldots & \ldots & \ldots & \ldots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \ldots & c & s & \ldots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \ldots & -s & c & \ldots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \ldots & 0 & 0 & \ldots & 1 \end{bmatrix},$$

é uma *matriz de Givens*. O valor c está na diagonal nos índices (i,i) e (j,j), $i<j$. O valor s está na posição (i,j) e o valor $-s$ na posição (j,i). Os elementos restantes da diagonal são 1 e os elementos fora da diagonal são 0.

Queremos que uma matriz de Givens seja ortogonal ($J(i,j,c,s)^T J(i,j,c,s) = I$), e claramente suas colunas são ortogonais. Basta então que tenham comprimento unitário, ou seja, que se tenha $c^2+s^2 = 1$. Para visualizar isso, considera a matriz de Givens

$$J(1,3,c,s) = \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix}.$$

Computa o produto $J(1,3,c,s)^T J(1,3,c,s)$ e verifica que $c^2+s^2 = 1$ é necessário para que $J(1,3,c,s)$ seja ortogonal:

$$J(1,3,c,s)^T J(1,3,c,s) = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix} \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix} = \begin{bmatrix} c^2+s^2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & c^2+s^2 \end{bmatrix}.$$

Usando a identidade trigonométrica $\cos^2\theta + \sin^2\theta = 1$, podemos tomar $c = \cos\theta$ e $s = \sin\theta$ e obter uma *matriz de rotação de Givens*, denotada por $J(i,j,\theta)$. No $\mathbb{R}^2$, a matriz de Givens $J(1,2,\theta)$ é $\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$.

O algoritmo da decomposição QR de Givens depende de zerar elementos em posições específicas de uma matriz. Seja $x = [x_1\ x_2\ \cdots\ x_{n-1}\ x_n]^T$ e calcula o produto $J(i,j,c,s)x$.

$$J(i,j,c,s)x = \begin{bmatrix} 1 & 0 & 0 & \dots & \dots & & \dots & \dots & 0 \\ 0 & 1 & 0 & \dots & \dots & & \dots & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & & \vdots \\ 0 & 0 & 0 & \dots & c & \dots & s & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & & \vdots \\ 0 & 0 & 0 & \dots & -s & \dots & c & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & & \vdots \\ 0 & 0 & 0 & \dots & 0 & \dots & \dots & \dots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_i \\ \vdots \\ x_j \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ cx_i + sx_j \\ \vdots \\ -sx_i + cx_j \\ \vdots \\ x_n \end{bmatrix}. \tag{12}$$

Nota que o produto $J(i,j,c,s)x$ altera somente as componentes i e j do vetor x. Então, se quisermos zerar a componente x_j, $i < j$, podemos reduzir o problema de encontrar c e s para o caso bidimensional. Toma a matriz de Givens $\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$ que rotaciona o vetor $\begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$ sobre o eixo x, zerando a coordenada y. Do plano cartesiano, é fácil ver que $\cos\theta = x/\sqrt{x^2+y^2}$ e $\sin\theta = y/\sqrt{x^2+y^2}$, então

$$\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & \frac{y}{\sqrt{x^2+y^2}} \\ -\frac{y}{\sqrt{x^2+y^2}} & \frac{x}{\sqrt{x^2+y^2}} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \sqrt{x^2+y^2} \\ 0 \end{bmatrix}.$$

Generalizando o argumento, para zerar x_j, escolha $i < j$ e calcula os valores

$$c = \frac{x_i}{\sqrt{x_i^2+x_j^2}}, \quad s = \frac{x_j}{\sqrt{x_i^2+x_j^2}}.$$

Faz $J(i,j,c,s) = I$ seguido de

$$\begin{aligned} J(i,i,c,s) &= J(j,j,c,s) = c \\ J(i,j,c,s) &= s \\ J(j,i,c,s) &= -s \end{aligned}$$

e computa $J(i,j,c,s)x$.

Segue-se que dado um vetor $x = [x_1 \ x_2 \ \dots \ x_n]^T$, podemos usar um produto de matrizes Givens para zerar os $n-i$ elementos de x abaixo da entrada x_i. Para zerar x_{i+1}, computa $J(i,i+1,c_{i+1},s_{i+1})x = \overline{x_{i+1}}$. Para zerar x_{i+2}, computa $J(i,i+2,c_{i+2},s_{i+2})\overline{x_{i+1}} = \overline{x}_{i+2}$ e continua o processo até $J(i,n,c_n,s_n)\overline{x_{n-1}} = \overline{x_n}$. Em suma, o produto

$$J(i,n,c_n,s_n)\dots J(i,i+2,c_{i+2},s_{i+2})J(i,i+1,c_{i+1},s_{i+1})x$$

transforma x num vetor da forma $[x_1 \ x_2 \ \dots \ x_{i-1} \ * \ 0 \ \dots \ 0]^T$.

Observação. As operações com matrizes de Givens podem ser feitas implicitamente, isto é, não é necessário construí-las.

Se $J(i,j,c,s)$ é uma matriz $m \times m$ e A é uma matriz $m \times n$, o produto $J(i,j,c,s)A$ altera somente as linhas i e j. De (12), vemos que as linhas i e j, $i < j$, têm forma

i: $ca_{i1} + sa_{j1} \ ca_{i2} + sa_{j2} \ \dots \ ca_{in} + sa_{jn}$

j: $ca_{j1} - sa_{i1} \ ca_{j2} - sa_{i2} \ \dots \ ca_{jn} - sa_{in}$

Isso faz com que o produto $J(i,j,c,s)A$ seja computado em apenas $6n$ operações. O algoritmo abaixo faz a multiplicação de A por uma matriz de Givens, dados os parâmetros c e s.

Listing 9: Multiplicação de uma matriz por uma matriz de Givens

```
function A = givensmul(A,i,j,c,s)
// multiplication of m x n matrix A by Givens matrix
// input: matrix A, indices i and j, Givens parameters c and s
// output: m x n matrix J(i,j,c,s)A

a = A(i,:);
b = A(j,:);

A(i,:) = c*a + s*b;
A(j,:) = -s*a + c*b;
endfunction
```

O cálculo dos parâmetros c e s pode ter problemas de *overflow* ou *underflow*. Uma maneira de minimizar esses erros é normalizar as entradas x_i e x_j, conforme o seguinte algoritmo.

Listing 10: Cálculo dos parâmetros de Givens

```
function [c,s] = givensparms(x_i,x_j)
    // normalize the entries x_i or x_j to minimize overflow/underflow
        errors
    // input: x_i and x_j, j>i, of vector x
    // output: Givens parameters c and s for x

    if x_j == 0 then
        c=1;
        s=0;
    elseif abs(x_j) > abs(x_i) then
        t=x_i/x_j; // make t such that 0<t<1
        s=1/sqrt(1+t^2);
        c=s*t;
    else
```

```
            t=x_j/x_i;
            c=1/sqrt(1+t^2);
            s=c*t;
        end
    endfunction
```

Definição 2. Uma matriz $A_{m\times n} = [a_{ij}]$ é *triangular-superior* se $a_{ij} = 0$ para $i > j$. Em outras palavras, A é triangular-superior se os elementos abaixo da diagonal principal são nulos.

O algoritmo abaixo realiza a decomposição QR de Givens.

Listing 11: Decomposição QR de Givens

```
function [Q,R] = givensqr(A)
    // compute the QR decomposition of A
    // input: m x n matrix A
    // output: m x n orthogonal matrix Q
    // and m x n upper-triangular matrix R
    [m,n] = size(A);
    Q = eye(m,m); // initialize Q = I

    for i = 1:min(m-1,n)
        for j = i+1:m // run through column i
            [c,s] = givensparms(A(i,i),A(j,i));
            A = givensmul(A,i,j,c,s);
            Q = givensmul(Q,i,j,c,s);
        end
    end
    R = A;
    Q = Q';
endfunction
```

2 Reflexões de Householder

As reflexões de Householder são uma alternatiiva às rotações de Givens para computar a decomposição QR de uma matriz. Uma rotação de Givens zera um elemento de cada vez, e é preciso de uma sequência de rotações para transformar uma matriz em forma triangular-superior. A reflexão de Householder, por outro lado, consegue zerar todos os elementos abaixo de um elemento da diagonal a_{ii} com uma única multiplicação, e por isso seu algoritmo para a decomposição QR requer cerca de dois-terços do número de *flops* do algoritmo de Givens. Entretanto, é importante notar que rotações de Givens são paralelizáveis (ao contrário das reflexões de Householder); além disso, em alguns casos, pode ser mais eficiente eliminar uma entrada por vez, situação que é mais adequada ao algoritmo de Givens.

Definição 3. Uma *reflexão de Householder* (ou *transformação de Householder*) H_u é uma transformação que reflete um vetor u num hiperplano no $\mathbb{R}^n$. A transformação tem a forma

$$H_u = I - 2\frac{uu^T}{u^Tu}, \tag{13}$$

onde u é um vetor não nulo. Evidentemente, H_u é uma matriz quadrada de ordem n, pois uu^T é uma matriz $n \times n$ e u^Tu é um escalar.

Aplicar uma transformação de Householder a um vetor $u \in \mathbb{R}^n$ resulta em $-u$. Consequentemente, $H_u(cu) = -cu$ para todo escalar c. De fato,

$$H_u(u) = \left(I - 2\frac{uu^T}{u^Tu}\right)u = u - 2\frac{uu^Tu}{u^Tu} = u - 2u = -u.$$

Sendo w um vetor no $\mathbb{R}^n$, seja $v = w - \text{proj}_u(w)$. Sabemos que v é ortogonal a u, o que implica que $w = c_1u + c_2v$, donde $c_1 = \langle w, u\rangle/\langle u, u\rangle$ e $c_2 = \langle w, v\rangle/\langle v, v\rangle$. Afirmamos que $H_u(w)$ é uma reflexão de w em relação ao hiperplano ortogonal a u. Com efeito,

$$\begin{aligned}
H_u(c_1u + c_2v) &= \left(I - 2\frac{uu^T}{u^Tu}\right)(c_1u + c_2v) \\
&= c_1u + c_2v - 2\frac{uu^T}{u^Tu}(c_1u + c_2v) \\
&= -c_1u + c_2v - 2c_2\frac{uu^T}{u^Tu}v \\
&= -c_1u + c_2v - 2c_2\frac{u\langle u, v\rangle}{\langle u, u\rangle} \\
&= -c_1u + c_2v
\end{aligned}$$

Proposição 1. *Seja H_u uma reflexão de Householder com $u \in \mathbb{R}^n$. Então*

1. *H_u é simétrica.*
2. *H_u é ortogonal.*
3. *$H_u^2 = I$.*
4. *$H_uv = v$ se $\langle v, u\rangle = 0$.*

Demonstração.

1. $H_u^T = \left(I - 2\frac{uu^T}{u^Tu}\right)^T = I - 2\frac{uu^T}{u^Tu} = H_u$.
2. Seja $\beta \stackrel{\text{def}}{=} 2/(u^Tu)$. Então $H_u^TH_u = (I - \beta uu^T)(I - \beta uu^T) = I - 2\beta uu^T + \beta^2u(u^Tu)u^T = I - 2\left(\frac{2}{u^Tu}\right)uu^T + \left(\frac{4}{(u^Tu)^2}\right)u(u^Tu)u^T = I$.
3. $H_u^2 = H_uH_u = I$.
4. Se $\langle v, u\rangle = 0$, então $H_uv = \left(I - 2\frac{uu^T}{u^Tu}\right)v = v - 2\frac{u\langle u, v\rangle}{\langle u, u\rangle} = v$.

□

Uma única reflexão de Householder pode zerar todos os elementos $a_{i+1,i}, a_{i+2,i}, \ldots, a_{mi}$ abaixo da entrada a_{ii} de uma matriz A. Aplicando uma sequência de reflexões de Householder, podemos transformar A numa matriz triangular-superior, ou seja, queremos transformar

$$A = \begin{bmatrix}
X & X & X & X & X & X & \ldots & X \\
X & X & X & X & X & X & \ldots & X \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ldots & \vdots \\
X & X & X & X & X & X & \ldots & X \\
X & X & X & X & X & X & \ldots & X
\end{bmatrix}$$

na forma

$$H_u A = \begin{bmatrix} X & X & X & X & X & X & \dots & X \\ 0 & X & X & X & X & X & \dots & X \\ 0 & 0 & X & X & X & X & \dots & X \\ 0 & 0 & 0 & X & X & X & \dots & X \\ 0 & 0 & 0 & 0 & X & X & \dots & X \end{bmatrix}.$$

Começamos zerando as entradas abaixo de a_{11}. Seja x a primeira coluna de A. Então $H_u x$ será a primeira coluna de $H_u A$. Se $x \neq ke_1 = [k\ 0\ \dots\ 0]^T$, buscamos um vetor u tal que $H_u x$ tenha zeros por toda parte exceto no índice $(1,1)$. Os elementos nas colunas restantes serão alterados, mas não nos preocupamos com isso.

A escolha do vetor u é melhor entendida geometricamente. Sabemos que $H_u x$ reflete x através do plano perpendicular a u, e que $\|H_u x\|_2 = \|x\|_2$ porque H_u é ortogonal. Então buscamos u tal que $H_u x = \pm\|x\|_2 e_1$. Em outras palavras, queremos refletir x através dum hiperplano que bisecta o ângulo entre x e e_1. Isso pode ser feito tomando $u = x \pm \|x\|_2 e_1$[7]. O sinal em $u = \pm\|x\|_2 e_1$ é escolhido para evitar cancelamento catastrófico. Como

$$[x_1\ x_2\ \dots\ x_{m-1}\ x_m]^T \pm \|x\|_2 e_1 = [x_1 \pm \|x\|_2 x_2\ \dots\ x_{m-1}\ x_m]^T,$$

apenas a primeira componente de u é afetada pelo sinal escolhido. Escolhemos o sinal de modo que uma adição seja feita.

$$u = \begin{cases} x + \|x\|_2 e_1, & x_1 > 0 \\ x - \|x\|_2 e_1, & x_1 < 0 \\ x + \|x\|_2 e_1, & x_1 = 0 \end{cases}$$

Outro possível problema é o overflow ou underflow no cálculo de $\|x\|_2$. Para minimizar esse risco, podemos normalizar x antes de calcular u e depois multiplicar u por $\|x\|_2$.

1. $\text{colmax} = \max(|x_1|, |x_2|, \dots, |x_m|)$
2. $\overline{x} = x/\text{colmax}$
3. $\|x\|_2 = \text{colmax}\|\overline{x}\|_2$

Lema. *Se H_u é uma reflexão de Householder e k é uma constante, então $H_{ku} = H_u$.*

Demonstração. $H_{ku} = I - 2\dfrac{(ku)(ku)^T}{(ku)^T(ku)} = I - 2\dfrac{k^2 uu^T}{k^2 u^T u} = I - 2\dfrac{uu^T}{u^T u} = H_u.$ □

Assim como no caso das rotações de Givens, a multiplicação por uma reflexão de Householder H_u não exige construir a matriz. Se H_u é uma matriz de Householder $m \times m$, x é um vetor no $\mathbb{R}^m$, e A é uma matriz $m \times n$, então os produtos $H_u x$ e $H_u A$ podem ser computador por uma simples fórmula. Se H_u é $n \times n$, o mesmo se aplica para AH_u (usado para zerar linhas de A). Sendo $\beta = 2/(u^T u)$, temos

$$H_u x = \left(I - 2\frac{uu^T}{u^T u}\right) x = x - \beta u(u^T x) \tag{14}$$

$$H_u A = \left(I - 2\frac{uu^T}{u^T u}\right) A = A - \beta u(u^T A) \tag{15}$$

$$AH_u = A\left(I - 2\frac{uu^T}{u^T u}\right) = A - \beta(Au)u^T \tag{16}$$

[7] O resultado pode ser verificado algebricamente, basta verificar a igualdade $H_u x = \mp\|x\|_2 e_1$

Usando a fórmula implícita $A - \beta u(u^T A)$, fazemos uma multiplicação vetor-matriz seguida de uma multiplicação escalar-vetor para computar $H_u A$ em apenas $\mathcal{O}(4mn)$ operações. Se fôssemos construir H_u explicitamente, precisaríamos de $\mathcal{O}(m^2)$ operações para construir H_u e $\mathcal{O}(m^2 n)$ para computar $H_u A$, totalizando $\mathcal{O}(m^2 n)$ operações, o que é muito mais custoso computacionalmente.

O seguinte algoritmo zera todos os elementos abaixo da entrada a_{11} usando uma reflexão de Householder.

Listing 12: Zerando entradas na primeira coluna de uma matriz usando uma reflexão de Householder

```
function [A,u] = hzero1(A)
    // zero out all elements below a_11 in matrix A by a Householder
        reflection H_u
    // input: m x n matrix A
    // output: m x n matrix A implicitly premultiplied by H_u and
        Householder vector u that defines the hyperplane about which x is
         reflected

    x = A(:,1);
    colmax = max(abs(x));
    x = x/colmax; // normalize x to minimize cancellation errors
    colnorm = norm(x,2);
    u = x;

    // construct Householder vector u
    if u(1) >= 0 then
        u(1) = u(1) + colnorm;
    else
        u(1) = u(1) - colnorm;
    end

    // implicitly form H_uA
    unorm = norm(u,2);
    if unorm ~= 0 then
        beta = 2/(unorm^2);
    else
        beta = 0;
    end
    A = A - (beta*u)*(u'*A);
endfunction
```

Para zerar os elementos abaixo das diagonais restantes com listing 12, criamos implicitamente uma sequência de matrizes de Householder que atuem em blocos de submatrizes. Suponha que tenhamos zerado todas as entradas abaixo da diagonal do índice $(1,1)$ ao $(i-1, i-1)$, obtendo uma sequência de matrizes $A_1, A_2, \ldots, A_{i-1}$. Precisamos de uma reflexão de Householder que zere os elementos $\overline{a_{i+1,i}}, \ldots, \overline{a_{mi}}$ e altere apenas elementos da submatriz correspondente de tamanho $(m-i+1, m-i+1)$. Então construímos $\tilde{H_u}$ com o vetor $x = [\overline{a_{ii}}\ \overline{a_{i+1,j}}\ \ldots\ \overline{a_{mi}}]^T$. Essa operação é realizada com o comando

```
[A(i:m,i:n), u] = hzero1(A(i:m,i:n)).
```

e repetida $k = \min(m-1, n)$ vezes para obter a matriz triangular-superior R. O processo assim resulta na fatoração QR da matriz A

$$\tilde{H}_k \tilde{H}_{k-1} \dots \tilde{H}_1 A = R \tag{17}$$

$$A = (\tilde{H}_1 \tilde{H}_2 \dots \tilde{H}_k)^T R \tag{18}$$

$$= (\tilde{H}_k \tilde{H}_{k-1} \dots \tilde{H}_1) R = QR, \tag{19}$$

onde $Q = \tilde{H}_1 \tilde{H}_2 \dots \tilde{H}_k$ é o produto de matrizes ortogonais, logo ortogonal. Inicializa $Q = I$ como matriz identidade $m \times m$ e computa $Q(1:m, 1:m) = IH_{u1}$, alterando as entradas em toda a matriz Q. Depois computa $Q(1:m, 2:m) = Q(1:m, 2:m)H_{u2}$, alterando as entradas em toda a matriz Q exceto as da primeira coluna, e assim por diante, até $Q(1:m, k:m) = Q(1:m, k:m)H_{uk}$. O algoritmo abaixo realiza a decomposição QR de Householder.

Listing 13: Decomposição QR de Householder

```
function [Q,R] = householderqr(A)
    // compute the QR decomposition of matrix A using Householder
        reflections
    // input: m x n matrix A
    // output: m x m matrix Q and m x n upper-triangular matrix R such
        that A=QR
    R = A;
    [m,n] = size(A);
    Q = eye(m,m);
    k = min(m-1,n);

    for i = 1:k
        [R(i:m,i:n),u] = hzero1(R(i:m,i:n));
        Q(1:m,i:m) = Q(1:m,i:m) - (2/norm(u,2)^2)*(Q(1:m,i:m)*u)*u';
    end
endfunction
```

A contagem de *flops* para o algoritmo de Householder é $\approx 2\left(mn^2 - \frac{n^3}{3}\right)$. O algoritmo de Givens requer $\approx 3n^2(2m - n)$ *flops*, ou seja, cerca de duas vezes mais do que o algoritmo de Householder no caso de matrizes $n \times n$.

VIII A Decomposição em Valores Singulares (SVD)

1 O Teorema SVD

2 Usando a SVD para determinar propriedades de uma matriz

3 Compressão de imagem usando a SVD

IX Problemas de mínimos quadrados

Nesta seção, estudamos o problema $Ax = b$ nos casos em que A é ou singular ou tem dimensão $m \times n$, $m \neq n$. Se $m > n$, existem mais equações que incógnitas, e o sistema é chamado de *sobredeterminado*. Geralmente, sistemas sobredeterminados não possuem solução. No caso $m < n$, existem mais incógnitas que equações, e o sistema é chamado de *indeterminado*. Nessa situação, normalmente existem infinitas soluções.

Como sistemas singulares, sobredeterminados ou indeterminados não levam a uma solução no sentido usual, queremos encontrar um vetor x tal que Ax é o mais próximo possível de b. Uma forma de fazer isso é encontrar x tal que o resíduo $r(x) = \|Ax - b\|_2$ é mínimo. Como a norma euclidiana, $\|\cdot\|_2$, de um vetor no $\mathbb{R}^n$ é dada por $\sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$, se queremos minimizar $\|Ax - b\|_2$, chamamos x a *solução de mínimos quadrados*. Encontrar a solução de mínimos quadrados de $Ax = b$ é conhecido como um *problema de mínimos quadrados linear*.

Definição 1. Dada uma matriz real $A_{m\times n}$ e um vetor real $b \in \mathbb{R}^m$, o problema de *mínimos quadrados* é encontrar um vetor $x \in \mathbb{R}^n$ tal que a função $r(x) = \|Ax - b\|_2$ seja minimizada. É possível que a solução x não seja única.

A ideia geométrica por trás do problema de mínimos quadrados é a seguinte. Suponha que $m > n$. Como $x \in \mathbb{R}^n$ e A é uma matriz $m \times n$, Ax é uma transformação linear de $\mathbb{R}^n$ em $\mathbb{R}^m$, e range(A) é um subespaço de $\mathbb{R}^m$. Dado qualquer $y \in$ range(A) existe $x \in \mathbb{R}^n$ tal que $y = Ax$. Se $b \notin$ range(A), considere o vetor $Ax - b$ que conecta as extremidades dos vetores Ax e b. Como $b \notin$ range(A), projeta b ortogonalmente no plano range(A) para obter um vetor $u \in$ range(A). Deve existir $x \in \mathbb{R}^n$ que satisfaz $Ax = u$. A distância entre os pontos, $\|Ax - b\|_2$, é a menor possível, então x é a solução que buscávamos.

O vetor $b - Ax$ é ortogonal a range(A), e como todo vetor em range(A) é uma combinação linear das colunas de A (vetores no $\mathbb{R}^m$), então deve ser o caso que $b - Ax$ é ortogonal a toda coluna de A. Isso significa que se a_i é a i-ésima coluna de A, então $\langle b - Ax, a_i\rangle = 0$ para todo i. Escrevendo isso em forma matricial, temos $A^T(b - Ax) = 0$, ou seja, $A^TAx = A^Tb$. As equações $A^TAx = A^Tb$ são chamadas de *equações normais* do problema de mínimos quadrados.

1 Existência e unicidade de soluções de mínimos quadrados

Definição 2. Uma matriz $A_{m\times n}$ tem *posto completo* (*full rank*) se rank$(A) = \min(m, n)$.

Lema. *Seja A uma matriz $m \times n$, $m \geq n$. Então A tem posto completo se e somente se A^TA é invertível.*

Demonstração. Provamos por contradição. □

Teorema 1. *1. Dada uma matriz $A_{m\times n}$, com $m \geq n$, e um vetor $b \in \mathbb{R}^m$, existe um vetor coluna $x \in \mathbb{R}^n$ tal que x é uma solução de mínimos quadrados de $Ax = b$ se e somente se x satisfaz as equações normais*

$$A^TAx = A^Tb.$$

2. A solução de mínimos quadrados de $Ax = b$ é única se e somente se A tem posto completo.

Teorema 2 (equação residual de mínimos quadrados). *Seja $r = b - Ax$. Então $A^Tr = 0$ se e somente se x é uma solução de mínimos quadrados de $Ax = b$.*

Demonstração. □

Se A é uma matriz $m \times n$, $m \geq n$, com posto completo, então A^TA é invertível, pelo 1, e o produto $(A^TA)^{-1}A^T$ está definido. Do 1, se A tem posto completo e x é uma solução para o problema de mínimos quadrados, então $A^TAx = A^Tb$ e logo

$$x = (A^TA)^{-1}A^Tb.$$

Definição 3. A matriz

$$A^\dagger = (A^TA)^{-1}A^T \tag{20}$$

é chamada de *pseudoinversa* ou *inversa generalizada de Moore-Penrose* de A. A solução para o problema de mínimos quadrados $Ax = b$ com posto completo é $x = A^\dagger b$.

Definição 4. Se A é uma matriz $m \times n$ com posto completo, então o número de condicionamento de A é dado por

$$\kappa(A) = \|A\|_2 \|A^\dagger\|. \tag{21}$$

2 Resolvendo problemas de mínimos quadrados sobredeterminados

Existem três métodos básicos para resolver problemas de mínimos quadrados sobredeterminados. O primeiro que estudaremos usa as equações normais $A^T Ax = A^T b$, que foi o método padrão por muitos anos. No entanto, usar a decomposição QR ou SVD costuma fornecer resultados melhores.

Sabe-se que $A^T A$ é simétrica. Considera o produto $x^T A^T Ax = (Ax)^T (Ax) = \|Ax\|_2^2$. Se supormos que A tem posto completo, então $x^T A^T Ax \neq 0$ para todo $x \neq 0$, o que implica que $A^T A$ é positiva definida. Portanto, $A^T A$ admite uma decomposição de Cholesky $A^T A = R^T R$, onde R é uma matriz triangular-superior. Para resolver as equações normais, procedemos da seguinte maneira:

Resolvendo as equações normais com a fatoração de Cholesky

a. Encontre a decomposição $A^T A = R^T R$ de Cholesky de $A^T A$.

b. Resolva $R^T y = A^T b$ para y usando substituição para frente.

c. Resolva $Rx = y$ para x usando substituição para trás.

3 Condicionamento de problemas de mínimos quadrados

X Autovalores e Autovetores

1 Teoremas de Gerschgorin

2 Método da potência para computar autovalor dominante

3 Iteração inversa de Hessenberg

XI Métodos Iterativos

1 Método de Jacobi

2 Método de Gauss-Seidel

3 Iteração SOR

4 Convergência dos métodos iterativos